Problem ID	Captured Flag	Steps (Summary)
P1 – nuclear_pasta	`fsuCTF{sp4ghett1_c0d3_15_a_d3fens3_mech4n1sm}`	Analyze convert() in binary, identify XOR 0xBB as sole meaningful operation, extract ct array from .data section, XOR each byte with 0xBB to recover flag
P2 – old_school_gauntlet	`fsuCTF{b3tt3r_7h4n_cry5t41_4rm0r}`	Bypass anti-debug checks (ptrace, /proc parent inspection, timing), reconstruct XOR loop with 4-byte key (dc 4e 72 9d) and 0x40 XOR, apply transformations to cipher blob
P3 – Upper Management	`fsuCTF{5ti11_b3tter_7h4n_73ch_supp0r7}`	Trace dynamic flag construction, reconstruct layered XOR and arithmetic transformations over embedded constants, recover final flag string

P1 – nuclear_pasta

The binary prompts the user for input and processes each byte through a function called convert(). After processing, it compares the transformed input against a constant byte array stored in the .data section.

Analysis of convert()

Despite its complex appearance, reversing convert() revealed that the only meaningful operation performed on each non-null byte was a single XOR:

```
if byte != 0:
    byte ^= 0xBB
```

This means the validation condition is:

```
input XOR 0xBB == ct
```

Therefore to recover the flag:

```
input = ct XOR 0xBB
```

Solution

1. Load the binary and locate the ct array in the .data section
2. Observe that convert() reduces to: if byte != 0, byte ^= 0xBB
3. XOR each byte of the ct array with 0xBB to recover the plaintext input
4. The result is the flag

Flag: **fsuCTF{sp4ghett1_c0d3_15_a_d3fens3_mech4n1sm}**

P2 – old_school_gauntlet

This binary implemented multiple anti-debugging checks before reaching the flag generation logic. Each check had to be understood and bypassed before the decryption routine could be analyzed.

Anti-Debug Checks

Three separate anti-debugging mechanisms were identified:

1. Parent process inspection via /proc/<pid>/exe to detect gdb, lldb, strace, or ltrace
2. ptrace(PTRACE_TRACEME) call to detect active tracing
3. A runtime timing check requiring execution to finish in under ~1 second

Flag Generation Logic

After bypassing the anti-debug checks, the flag generation routine was analyzed. It used an XOR-based decryption loop with the following components:

```
XOR key (4 bytes): dc 4e 72 9d
Additional XOR:    0x40
Input:             cipher blob embedded in binary
```

Reconstructing the loop logic and applying both XOR transformations to the cipher blob yielded the flag:

Flag: **fsuCTF{b3tt3r_7h4n_cry5t41_4rm0r}**

P3 – Upper Management

This challenge required analysis of how the program dynamically constructed a flag string at runtime rather than storing it statically in the binary.

Analysis

Through reverse engineering, the binary was found to use layered transformations to build the flag:

1. XOR operations over embedded constant values
2. Arithmetic transformations (additions/subtractions) applied at each step
3. Constants were scattered across the binary to obscure the final output

Solution

4. Trace the flag-building logic step-by-step through the binary
5. Identify each embedded constant and the transformation applied to it
6. Reconstruct the full sequence of XOR and arithmetic operations
7. Apply the reconstructed transformations to recover the final flag string

Flag: **`fsuCTF{5ti11_b3tter_7h4n_73ch_supp0r7}`**